

Lab 01 – Firmware Reverse Engineering – First Cut

Industrial Security (Master)

• Maps to Section 01

• 2–3 hours

LAB 01

🎯 Goal

End-to-end first cut: take a lawfully-sourced firmware image, identify its architecture and filesystem, find at least one hard-coded credential or weak crypto pattern, and produce a 1-page write-up suitable for a coordinated-disclosure email.

✂ Step Tooling

On a Linux VM (Debian / Ubuntu / Kali):

```
1 sudo apt install -y binwalk squashfs-tools cpio xxd file strings unzip
2 # Ghidra: download from https://ghidra-sre.org and install per docs
3 pip install --user firmware-mod-kit # optional
```

✂ Step 1 – Source the image

Pick one that is safe to handle:

- OpenWrt for a router model you own.
- Discontinued home-router firmware from the vendor archive.
- Vendor partner programme (Siemens Industry Online, Rockwell PCDC) if you have access.

Record the URL, the published hash, your local hash, and the date.

✂ Step 2 – Triage

```
1 file firmware.bin
2 binwalk -A firmware.bin      # architecture hints
3 binwalk -E firmware.bin      # entropy plot
4 binwalk -e firmware.bin      # extract everything
```

Note bootloader / kernel / filesystem offsets.

✂ Step 3 – Filesystem mount

```
1 cd _firmware.bin.extracted
2 unsquashfs *.squashfs      # or use the existing extraction
3 cat squashfs-root/etc/passwd
4 ls squashfs-root/usr/sbin
```

✂ Step 4 – Strings hunt

```
1 grep -rIE 'password|secret|token|admin' squashfs-root/etc squashfs-root/usr/
  sbin
2 strings -n 8 squashfs-root/usr/sbin/lighttpd | grep -iE 'pass|key'
```

 Step 5 – One Ghidra dive

Open the most interesting binary in Ghidra. Auto-analyse. Find one symbol that looks like authentication or update verification. Read the decompiled C and document one weakness.

 Deliverable

A 1-page write-up containing: source + hash, architecture, filesystem layout, one finding with reproduction steps, CWE classification, and the disclosure email you would send.

 Caution

Do not test findings against any device you do not own. Do not publish exploit code before vendor disclosure. Keep proof-of-concept code in a private repository.